

IFT542 — WEB APPLICATION SECURITY

**Security Assessment and Hardening of the Student
Registration Web Application**

Technical Report

Name: [Your Full Name]

Matric Number: [Your Matric No.]

Course: IFT542

Date: July 2026

Table of Contents

Introduction

This report documents the security assessment and hardening of a prototype Student Registration Web Application built with a PHP 8.1 / MySQL 8.0 / Apache stack (Bootstrap 5 frontend), developed and tested exclusively on a local XAMPP instance using fictitious data. The application supports student login, profile updates, course registration, document upload, and administrative management of courses and enrolments.

The work is organised into three tasks: a STRIDE threat model and risk assessment of the application (Task 1); remediation of an unsafe authentication flow and SQL injection vulnerability (Task 2); and implementation of defences against XSS, CSRF, SSRF, and security misconfiguration, together with logging, incident response, and an ethics statement (Task 3). All testing was authorised-lab-only: no FUT Minna system, public website, or third-party service was scanned or probed at any point.

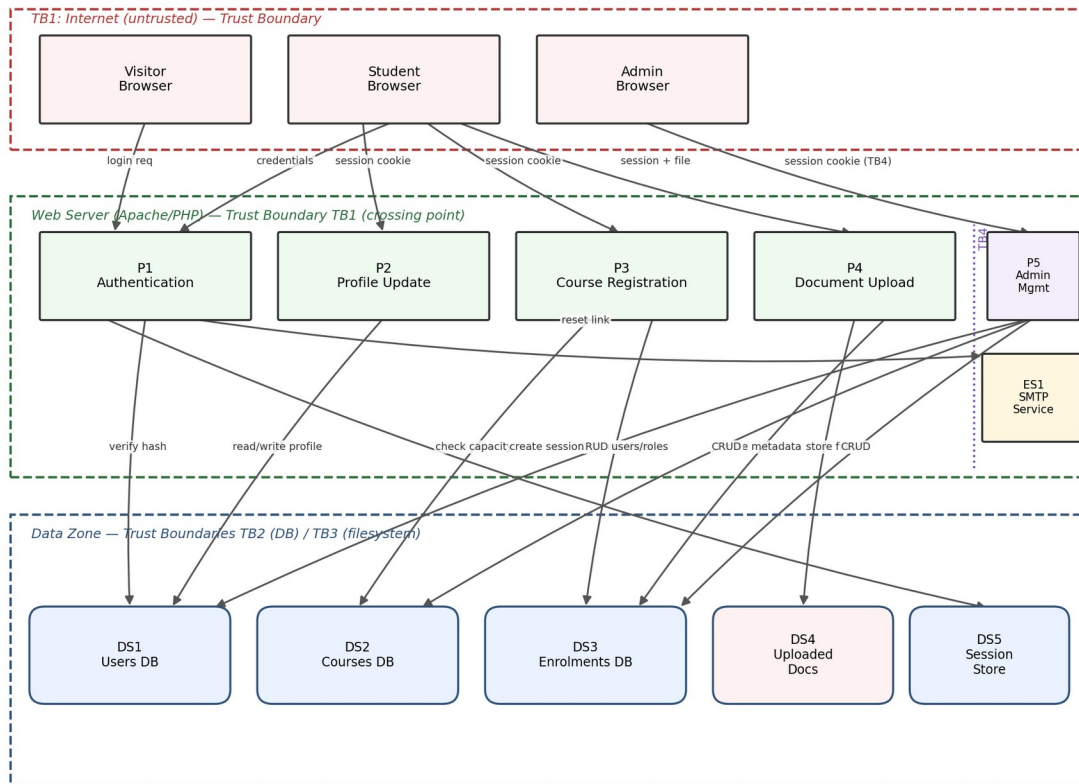
Task 1: STRIDE Threat Model and Risk Assessment

1.1 System Description

The application scope for threat modelling comprises three actor types (unauthenticated visitor, student, admin), five server-side processes (Authentication, Profile Update, Course Registration, Document Upload, Admin Management), five data stores (Users DB, Courses DB, Enrolments DB, Uploaded Documents, Session Store), one external service (SMTP mail), and five trust boundaries: Internet ↔ Web Server, Web Server ↔ Database, Web Server ↔ Filesystem, Student-privilege ↔ Admin-privilege, and Web Server ↔ External SMTP.

1.2 Data-Flow Diagram

The diagram below shows all actors, processes, data stores, the external mail service, and the five trust boundaries crossed by data in the application.

Student Registration Web Application — Data-Flow Diagram with Trust Boundaries

1.3 STRIDE Worksheet

Eight threats were identified across the six STRIDE categories (two per category for Tampering and Information Disclosure), exceeding the minimum of six required.

#	STRIDE	Threat	Component	Data Flow
T1	Spoofing	Attacker guesses/reuses leaked credentials to log in as a student	P1 Auth	Browser→P1→DS1
T2	Tampering	Login query concatenates raw input, enabling SQL injection	P1 Auth	Browser→P1→DS1 (TB2)
T3	Tampering	No CSRF token on course registration; forged cross-site request	P3 Course Reg.	Browser→P3→DS3
T4	Repudiation	No audit trail of who registered/dropped a course or changed a grade	P3, P5	P3/P5→DS3
T5	Info. Disclosure	Verbose login errors enable account enumeration; stack traces leak schema	P1 Auth	P1→Browser
T6	Info. Disclosure	URL-import feature performs unrestricted server-side fetch (SSRF)	P4 Upload	P4→internal network
T7	Denial of Service	No rate limiting allows credential-stuffing scripts to exhaust resources	P1 Auth	Browser→P1
T8	Elev. of Privilege	Profile update trusts client-supplied user_id/role, bypassing	P2 / P5	Browser→P2→DS1

#	STRIDE	Threat	Component	Data Flow
		TB4		

1.4 Risk Register

Likelihood and impact are scored 1–5; Risk = Likelihood × Impact.

#	Threat	Likelihood	Impact	Score	Priority
T2	SQL injection in login	4	5	20	1
T8	Privilege escalation via profile update	3	5	15	2
T6	SSRF via document preview	3	4	12	3
T3	CSRF on course registration	4	3	12	3 (tie)
T1	Credential stuffing / weak-password spoofing	3	4	12	3 (tie)
T7	DoS via unthrottled login attempts	4	2	8	4
T5	Account enumeration / verbose errors	4	2	8	4 (tie)
T4	Repudiation — no audit trail	2	3	6	5

1.5 Top Three Priority Risks — Justification

Priority 1 — T2 (SQLi, score 20): highest likelihood and maximum impact combined — login forms are the most-probed entry point, and a successful injection can bypass authentication entirely or dump/modify the Users table, threatening confidentiality, integrity, and availability at once.

Priority 2 — T8 (Elevation of privilege, score 15): equally severe impact to T2 — any student who discovers this flaw gains full admin control, undermining the core authorization boundary (TB4) the whole application depends on.

Priority 3 — T6 (SSRF, score 12, ranked ahead of the other 12-point ties): the only threat here able to escape the application boundary entirely and touch the underlying host/network (e.g., cloud metadata, internal-only services), giving an attacker a foothold beyond the web app itself. CSRF and credential stuffing (also scored 12) remain contained to in-app actions and are comparatively cheaper to mitigate.

1.6 Controls and Residual Risk

#	Threat	Control(s)	Residual Score	Accepted?
T2	SQLi	Prepared statements; email format validation	5	Yes — see 1.7
T8	Privilege escalation	Server-side role check; identity from session only	5	Yes
T6	SSRF	Destination allowlist; block private/loopback/metadata ranges	3	Yes
T3	CSRF	Per-session token; SameSite=Lax cookies	3	Yes
T1	Credential stuffing	Bcrypt hashing; account lockout; optional admin MFA	8	Yes
T7	DoS on login	IP + account rate limiting; exponential backoff	4	Yes
T5	Account enumeration	Generic errors; display_errors off in production	2	Yes
T4	Repudiation	Structured audit log (who/what/when)	3	Yes

1.7 Justification for Accepted Residual Risk

All residual scores fall to 8 or below after controls are applied. The remaining risk in each case stems from factors partially outside the application's direct control — for example, users reusing passwords across other sites, or a future developer omitting a role check on a new endpoint. These are accepted rather than eliminated entirely because the cost of doing so (for example, mandatory hardware MFA for every account) would be disproportionate to a final-year prototype's actual threat exposure, and the detective controls already in place (audit logging, account lockout) provide an acceptable compensating safety net.

Task 2: Secure Authentication and SQL Injection Remediation

2.1 Locating the Unsafe Query

File: login.php (baseline, preserved as login_vulnerable.php). Affected data flow: Browser → P1 Authentication → DS1 Users DB.

```
$email = $_POST['email'] ?? '';
$sql = "SELECT id, full_name, email, password_hash, role
      FROM users
      WHERE email = '$email'";
$result = $conn->query($sql);
```

This pattern is unsafe because the value from `$_POST['email']` is inserted directly into the SQL text through string interpolation. The database cannot distinguish data the user typed from SQL the developer wrote — both arrive as one string. An input such as `' OR '1'='1` closes the intended string literal early and appends an always-true condition, turning “find the one row matching this email” into “return the first row in the table,” which can bypass authentication for any account.

2.2 The Fix — Parameterized Query

```
$stmt = $conn->prepare(
    'SELECT id, full_name, email, password_hash, role, failed_attempts, locked_until
    FROM users WHERE email = ?'
);
$stmt->bind_param('s', $email);
$stmt->execute();
$result = $stmt->get_result();
```

With `mysqli::prepare()`, the SQL text (`WHERE email = ?`) is compiled by the MySQL server before any user input is attached. The placeholder is a fixed slot in that already-compiled query plan; `bind_param('s', $email)` sends the value afterward, explicitly tagged as a string parameter. MySQL never re-parses that value as SQL syntax, so an attacker's input can change only the data being searched for, never the query's structure.

2.3 Additional Fixes Bundled With the Query Fix

Issue	Before	After
Password check	Plaintext comparison fallback alongside <code>password_verify</code>	Only <code>password_verify()</code> — no plaintext path
Input validation	None — raw <code>\$_POST['email']</code> used directly	<code>filter_var(\$email, FILTER_VALIDATE_EMAIL)</code> before any DB call
Error messages	Distinct messages reveal account existence	Single generic “Invalid email or password.”
Error detail to browser	DB errors could bubble up	<code>display_errors</code> off in production; errors logged server-side only

2.4 Additional Authentication Controls

- Account lockout: after 5 consecutive failed attempts, `users.locked_until` is set 15 minutes into the future; even a correct password is rejected while locked.
- IP-based rate limiting: a `login_attempts` table blocks further attempts after 10 attempts from one IP within 5 minutes, regardless of which account is targeted.

- Session-ID regeneration: `session_regenerate_id(true)` is called immediately after a successful login, preventing session fixation.

2.5 Test Evidence

Automated tests (`tests/task2_tests.php`) verify: valid credentials succeed; wrong password is rejected with a generic message; an unknown account returns the identical generic message (no enumeration); a classic ' OR '1'='1 payload and a payload embedded in an email-shaped string are both rejected; sampled `password_hash` values match the bcrypt format (`$2y$/$2b$, 60 characters`); and account lockout engages after five consecutive failures.

2.6 Database Evidence

Query used to demonstrate hashed storage without exposing real credentials:

```
SELECT email, LEFT(password_hash, 7) AS hash_prefix, LENGTH(password_hash) AS hash_len
FROM users;
```

Expected output shows a `hash_prefix` of `$2y$12$` or `$2b$12$` and a `hash_len` of 60 for every row, confirming no plaintext or reversible value is stored, without printing the full hash or any password.

Task 3: Web Defences, Incident Response and Ethics

3.1 XSS — Contextual Output Encoding and CSP

Field protected: the student profile “bio” field (profile.php, compared to profile_vulnerable.php).

```
// Before (vulnerable – stored XSS):
<div class="border p-2 bg-white"><?= $bio ?></div>

// After (fixed):
<div class="border p-2 bg-white"><?= htmlspecialchars($bio, ENT_QUOTES) ?></div>
```

htmlspecialchars() with ENT_QUOTES converts <, >, &, ", and ' into HTML entities, so a saved <script> tag renders as inert text rather than being parsed as an element. As defence in depth, a restrictive Content-Security-Policy is set on every page (includes/security_headers.php): script-src 'self' blocks any inline or third-party script from executing even if an encoding gap were ever missed, and frame-ancestors 'none' prevents clickjacking.

3.2 CSRF — Token and SameSite

Protected action: course registration (courses.php, compared to courses_vulnerable.php).

```
// includes/csrf.php
function csrf_token(): string { ... bin2hex(random_bytes(32)) ... }
function require_csrf($actorEmail = ''): void {
    if (!verify_csrf_token($_POST['csrf_token'] ?? null)) {
        log_event('csrf_validation_failed', ...);
        http_response_code(403);
        die('Request rejected...');
    }
}
```

The token is a random, session-bound 256-bit value checked with a constant-time comparison (hash_equals), applied to every state-changing form: course registration, profile update, document upload, and admin course creation. Session cookies also use SameSite=Lax as an independent second layer.

3.3 SSRF — Allowlist and Private-Range Blocking

Hardened feature: the “import document from URL” option on the upload page (upload.php, compared to upload_vulnerable.php).

The fix in includes/url_safety.php layers three checks: only http/https schemes are permitted; only pre-approved hostnames may be requested; and the hostname is resolved and the actual IP is re-checked against private/loopback/link-local/reserved ranges (FILTER_FLAG_NO_PRIV_RANGE | FILTER_FLAG_NO_RES_RANGE), which also defeats DNS rebinding. Every blocked attempt is logged.

3.4 Security Misconfiguration

- Debug/defaults: display_errors and error_reporting disabled in production mode; no diagnostic/test scripts ship in the app tree.
- Secrets: no API keys or credentials hardcoded; config.php uses local placeholders, flagged in the README.

- Security headers: CSP, X-Content-Type-Options: nosniff, X-Frame-Options: DENY, Referrer-Policy, Permissions-Policy, conditional HSTS.
- Server hardening: .htaccess disables directory listing, hides the Apache version banner, and blocks direct access to db/, includes/, and tests/.
- Dependency status: Bootstrap pinned to 5.3.3 from a single allowlisted CDN origin; no other third-party runtime dependency is used.

3.5 Logging

log_event() writes actor email, action, a short detail string, source IP, and timestamp to the audit_log table for login successes/failures/lockouts/rate-limits, authorization denials, CSRF failures, SSRF blocks, validation rejections, and all state-changing actions (registration, profile update, uploads, course management) — never a password, token, or session ID. Viewable via admin/logs.php.

3.6 Incident Response Runbook (Summary)

A one-page runbook covers all six stages: Preparation (always-on logging, backups, designated Incident Lead/Escalation Contact); Identification (monitoring the audit log for spikes in failures/blocks); Containment (automatic account lockout and IP rate limiting, manual IP blocking if needed); Eradication (root-cause fix, credential rotation, removal of attacker-written files); Recovery (restore from backup if needed, re-run the test suite, re-enable access); and Lessons Learned (post-incident review, STRIDE register update, new regression test). The full runbook and a filled sample incident record (using the Task 2 test-suite activity as simulated data) are provided as submission evidence.

3.7 Ethics Statement

All testing described in this report was performed exclusively against a local XAMPP instance (localhost) using fictitious seed data (@example.test addresses, invented names and matric numbers). No FUT Minna system, public website, or third-party service was scanned, probed, or tested at any point. No real credentials, personal data, or production systems were used. This work is submitted solely for the IFT542 coursework assessment.

Conclusion

This assessment modelled eight realistic threats against the Student Registration Web Application across all six STRIDE categories, prioritised them using a transparent likelihood-times-impact risk score, and implemented and tested concrete remediations for the three highest-priority risks (SQL injection, privilege escalation, and SSRF) as well as CSRF, XSS, and security misconfiguration. Defence-in-depth was applied throughout — for example, CSP as a backstop to output encoding, and SameSite cookies alongside CSRF tokens — and every security-relevant event is captured in a structured audit log to support the incident-response process documented above. All work was conducted under the authorised-lab restriction using fictitious data only.